

Epiplexity with lightweight learning algorithms

Experimental design for MLP, CNN and DiT setups

September 2026

Bounded observers

Shannon entropy and Kolmogorov complexity place no limit on the computation of the observer. Epiplexity measures information relative to a time bound (Finzi et al., 2026, Def. 7–8).

Definition (T -time probabilistic model)

Fix a prefix-free universal Turing machine $\mathcal{U}$ and a time bound $T : \mathbb{N} \rightarrow \mathbb{N}$. A program P is a T -time probabilistic model over $\{0, 1\}^n$ if, within $T(n)$ steps of $\mathcal{U}$, it

- outputs the probability $P(x)$ of any $x \in \{0, 1\}^n$;
- outputs a sample $x \sim P$ from a string of random bits.

$\mathcal{P}_T$ is the set of such programs; $|P|$ is the length of P in bits.

Logarithms are base 2.

Epiplexity and time-bounded entropy

Definition

For a random variable X on $\{0, 1\}^n$, let

$$P^* = \arg \min_{P \in \mathcal{P}_T} \{|P| + \mathbb{E}[-\log P(X)]\},$$

with ties broken by the shortest program. Then

$S_T(X) := |P^*|$ epiplexity: structure,

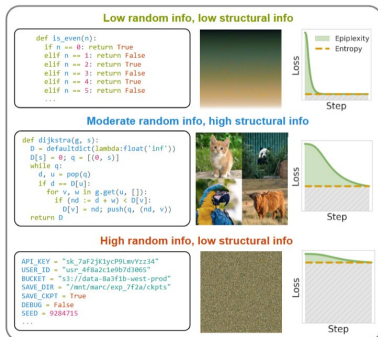
$H_T(X) := \mathbb{E}[-\log P^*(X)]$ time-bounded entropy: randomness,

$\text{MDL}_T(X) := S_T(X) + H_T(X)$ total description length.

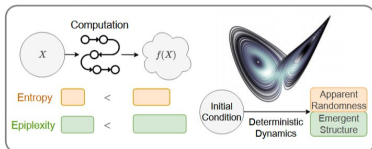
- Uniform random bits: $S_T = O(1)$ and $H_T \geq n - O(1)$.
- S_T depends on T and on $\mathcal{P}_T$; it need not grow with T .

Random and structural information

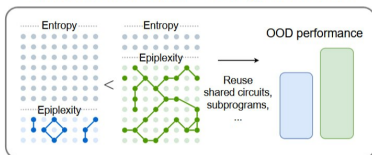
Random vs structural information



Information can be created by computation



Structural information $\rightarrow$ OOD generalization



Left: a network trained on three kinds of data. The final loss approximates H_T (dashed); the area above it approximates S_T (green).

Right: computation can create both kinds of information; learned structure transfers out of distribution (OOD).

Source: Fig. 1 of Finzi et al. (2026).

From the definition to a measurement

A *setup* a fixes an architecture, an optimiser, a training schedule and, for a diffusion transformer (DiT), a sampler.

- Compute is in FLOPs. A network with N parameters, trained on D examples and evaluated on the D_X examples of X , needs $6ND + 2ND_X \leq T$.
- $\mathcal{P}_T^a$ is the set of models that setup a reaches within T .
- The code length $K(P)$ of a trained model P replaces $|P|$.

The measured epiplexity is $S_T^a(X) := K(\hat{P})$, where

$$\hat{P} = \arg \min_{P \in \mathcal{P}_T^a} \{K(P) + \mathbb{E}[-\log P(X)]\}.$$

We sweep N and D . Each checkpoint gives the point (FLOPs, $K + D_X \cdot \text{held-out loss}$); S_T^a is the model part on the lower convex hull at T (Finzi et al., 2026).

Estimators I: prequential code and EDL

A training run sees examples $z_0, \dots, z_{M-1}$, sampled i.i.d. Model P_i is trained on $z_0, \dots, z_{i-1}$; its loss is $\ell_i(z) = -\log P_i(z)$.

Prequential code (Finzi et al., 2026): send z_i in $\ell_i(z_i)$ bits, then train on it. The model part is the area of the loss curve above the final loss:

$$K_{\text{preq}} \approx \sum_{i < M} [\ell_i(z_i) - \ell_M(z_i)].$$

Excess description length (Donoway et al., 2026): the floor is the population loss $L(P) = \mathbb{E}_z[-\log P(z)]$ of the model P_{fin} at the end of training with compute T :

$$\text{EDL}_T = \mathbb{E} \sum_{i < M} \ell_i(z_i) - M L(P_{\text{fin}}).$$

The expectation is over the sample and its order. EDL is defined for one fixed training algorithm.

Estimator II: sequential code

Sequential coding transmits a student model (Qiu et al., 2026).

- A teacher P_i^t trains on real data. A student P_i^s trains only on samples of the teacher.
- Encoder and decoder share the student initialisation, update rule and random seed.
- Relative entropy coding sends one sample of P_i^t to a decoder that knows P_i^s in about $\text{KL}_i = \text{KL}(P_i^t \parallel P_i^s)$ bits (Kullback–Leibler divergence).

Over student steps i the code length is

$$K_{\text{req}} \leq \sum_i [\text{KL}_i + 2 \log(1 + \text{KL}_i) + \kappa], \quad \kappa < 5.21.$$

Both papers report $\sum_i \text{KL}_i$. The code contains only the disagreement of teacher and student; the entropy of the data does not enter.

What we test: the estimators, not the architectures

The measured value depends on the data X , the bound T and the setup

$$a \in \{\text{MLP, CNN, DiT + sampler}\}.$$

Two sources of change have to be told apart:

- *manipulations* inside a setup: example order, input permutation, sampler, bound T ;
- the *observer family*: hypothesis class and inductive bias.

The setup dependence is

$$\Delta_{ab}(X, T) = S_T^a(X) - S_T^b(X).$$

Goal: which part of $S_T^a(X)$ comes from X , which from a .

Benchmark: elementary cellular automata (ECA)

- A row of n cells takes values in $\{0, 1\}$. A rule $r \in \{0, \dots, 255\}$ sets each cell from itself and its two neighbours; F_r updates all cells at once.
- Task $d = (r, t)$: the input x is uniform on $\{0, 1\}^n$; the label is $y = F_r^t(x)$.
- The codes cover labels only. Every estimate is then a conditional epiplexity $S_T(Y | X)$: the definition with $-\log P(Y | X)$ in place of $-\log P(X)$.

Finzi et al. (2026) report, for transformers on 64 cells and $t = 48$:

Rule	54	15	30
S_T , bits	$\approx 5 \times 10^6$	$\approx 2 \times 10^5$	≈ 0

Rule 30 looks random to a bounded observer: its information is almost all H_T .

Setups and manipulations

	MLP	CNN	DiT
Parameters N	10^3 – 10^6	10^3 – 10^6	small $\rightarrow$ medium
Prior	weakest	locality	generative, iterative
Optimiser	SGD / Adam(W)	SGD / Adam(W)	Adam(W)
Manipulation	example order	input permutation	sampler, #steps

- *Example order*: the order of $z_0, \dots, z_{M-1}$. An MLP has no spatial prior.
- *Input permutation*: one fixed permutation of cells or pixels. Labels are unchanged; locality is removed.
- *Sampler*: the denoising schedule of the DiT; #steps is the number of network evaluations per sample.

Within one comparison the task and T are fixed.

Validation

For each task d , setup a , bound T and seed s we compute S_{preq} and S_{req} (the value S_T^a with $K = K_{\text{preq}}$ or K_{req}) and EDL_T .

- **Agreement:** Spearman ρ and Kendall τ between the task orderings of two estimators, within and across setups.
- **Setup dependence:** Δ_{ab} per task, averaged over seeds. A large Δ with a stable task order means the estimate partly measures the setup.
- **Seed variance:** the standard deviation over s , compared with the spread across setups.
- **Baselines:** final test loss, sample efficiency.

Deliverable: a map of where each estimator is setup-invariant.

Literature

- M. Finzi, S. Qiu, Y. Jiang, P. Izmailov, J. Zico Kolter, A. G. Wilson. *From Entropy to Epiplexity: Rethinking Information for Computationally Bounded Intelligence*. arXiv:2601.03220, 2026.
- S. Qiu, M. Finzi, Y. Zheng, K. Zhang, A. G. Wilson. *Requential Coding: Pushing the Limits of Model Compression with Self-Generated Training Data*. arXiv:2607.11883, 2026.
- E. Donoway, H. Joren, F. Roger, J. Leike. *Excess Description Length of Learning Generalizable Predictors*. arXiv:2601.04728, 2026.